



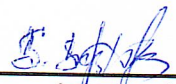
МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт кибербезопасности и цифровых технологий (ИКБ)
Базовая кафедра «Прикладное программирование»

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ
Проектно-технологическая практика

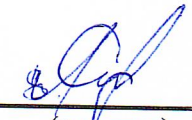
Тема практики: «Разработка алгоритмов машинного обучения перевода аналоговой информации в цифровую»
приказ Университета о направлении на практику от «27» февраля 2025 г. № 546-С

Отчет представлен к
рассмотрению:

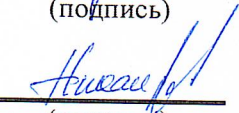
Студент группы БФБО-01-24 «29» 05 2026 г.  С. Сапармуратов
(подпись)

Отчет утвержден.
Допущен к защите:

Руководитель практики от
кафедры

«29» 05 2026 г.  С.П. Луцкан
(подпись)

Руководитель практики от
профильной организации

«29» 05 2026 г.  Л.Б. Никандров
(подпись)

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт кибербезопасности и цифровых технологий (ИКБ)
Базовая кафедра «Прикладное программирование»

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ПРОИЗВОДСТВЕННУЮ ПРАКТИКУ
Проектно-технологическая практика

Студенту 2 курса учебной группы БФБО-01-24
Сапармуратова Сапармурата

Место и время практики: АО «Метрогипротранс», 09 февраля 2026 г. по 30 мая 2026 г.

Должность на практике: практиканта

1. ЦЕЛЕВАЯ УСТАНОВКА: развитие способностей в области проектной деятельности путем применения современных методов и инструментальных средств прикладной информатики

2. СОДЕРЖАНИЕ ПРАКТИКИ:

2.1. Изучить: информационные и прикладные процессы работы информационной системы

2.2. Практически выполнить: применить современные инструментальные средства для оптимизации, улучшения и производительности информационной системы

2.3. Ознакомиться: с уровнем развития информационных и прикладных процессов информационной системы

3. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ: оформить презентационный материал

4. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в процессе практики рекомендуется использовать методические рекомендации по прохождению проектно-технологической практики

Заведующий кафедрой:

«09» февраля 2026 г.

СОГЛАСОВАНО:

Руководитель практики от кафедры

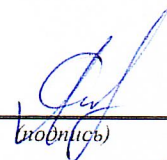
«09» февраля 2026 г.

Руководитель практики от профильной организации

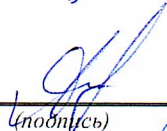
«09» февраля 2026 г.

Задание получил

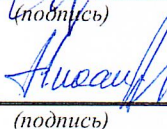
«09» февраля 2026 г.


(подпись)

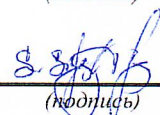
С.П. Луцкан


(подпись)

С.П. Луцкан


(подпись)

Л.Б. Никандров

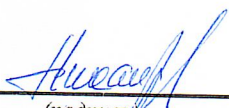

(подпись)

С. Сапармуратов

Проведенные инструктажи:

Охрана труда:

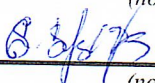
Инструктирующий


(подпись)

«09» февраля 2026 г.

Л.Б. Никандров

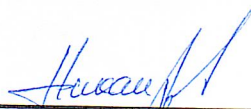
Инструктируемый


(подпись)

С. Сапармуратов

Техника безопасности:

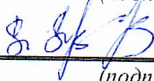
Инструктирующий


(подпись)

«09» февраля 2026 г.

Л.Б. Никандров

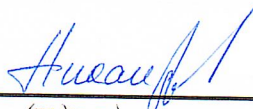
Инструктируемый


(подпись)

С. Сапармуратов

Пожарная безопасность:

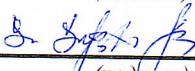
Инструктирующий


(подпись)

«09» февраля 2026 г.

Л.Б. Никандров

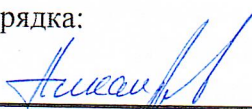
Инструктируемый


(подпись)

С. Сапармуратов

Правила внутреннего трудового распорядка:

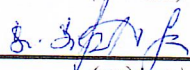
Инструктирующий


(подпись)

«09» февраля 2026 г.

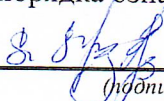
Л.Б. Никандров

Инструктируемый


(подпись)

С. Сапармуратов

С правилами внутреннего распорядка ознакомлен:


(подпись)

«09» февраля 2026 г.

С. Сапармуратов



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

РАБОЧИЙ ГРАФИК ПРОВЕДЕНИЯ

Проектно-технологическая практика

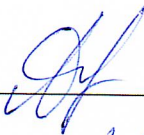
Студента С. Сапармурадова 2 курса группы БФБО-01-24 очной формы обучения, обучающегося по направлению подготовки 02.03.02 «Фундаментальная информатика и информационные технологии», профиль «Искусственный интеллект и машинное обучение».

Неделя	Сроки выполнения	Этап	Отметка о выполнении
1	09.02.2026	Подготовительный этап, включающий в себя организационное собрание	Выполнено
2	03.03.2026	Выполнение задания по практике в соответствии с выданным заданием студента.	Выполнено
3	17.04.2026	Подготовка отчета и презентационного материала по практике	Выполнено
4	29.05.2026	Защита отчета по производственной практике у руководителя практики.	Выполнено

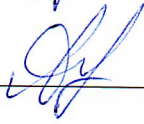
Содержание практики и планируемые результаты согласованы с руководителем практики от профильной организации

Согласовано:

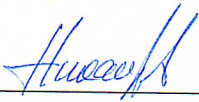
Заведующий кафедрой

 /С.П. Луцкан/

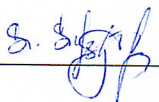
Руководитель практики от кафедры

 / С.П. Луцкан/

Руководитель практики от профильной организации

 /Л.Б. Никандров/

Обучающийся

 /С. Сапармурадов/

ОТЧЁТ

Проектно-технологическая практика

студента 2 курса учебной группы БФБО-01-24
Института кибербезопасности и цифровых технологий Университета РТУ МИРЭА
Сапармурата Сапармурадова

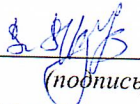
1. Практику проходил с 09.02.2026 г. по 30.05.2026 г. в АО «Метрогипротранс», практикант
(место прохождения практики и должность)
2. Задание на практику выполнил в полном объеме
(указать: в полном объеме или частично)

Не выполнены следующие задания:

(указать также причины невыполнения)

Подробное содержание выполненной на практике работы и достигнутые результаты: изучена структура организации и рабочего процесса, оптимизирован механизм передачи данных между компонентами системы, оптимизированы процессы синхронизации данных, сэкономлены серверные ресурсы организации, разработан интерфейс получения данных. Изучены подходы к реализации архитектуры современных приложений, исследованы компоненты современных приложений

Предложения по совершенствованию организации и прохождения практики:
предложений нет

Студент  (С. Сапармурадов)
(подпись)

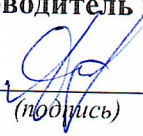
«30» мая 2026 г.

Заключение руководителя практики

Приобрел следующие профессиональные навыки: практикант продемонстрировал профессиональные умения и навыки, знание и понимание прикладной области, задач, требующих решения в прикладной области, современные подходы и средства решения прикладных задач разных классов, умение находить и работать с различными источниками информации по профессиональной деятельности, структурировать отчет с учетом тематики исследования
Проявил себя как: дисциплинированный ответственный специалист: соблюдал сроки календарного графика практики, регулярно отчитывался о проделанных этапах работ; за срок прохождения практики не получил ни одного замечания - проявляет инициативу, четко и в определенные сроки выполняет задания; в любой ситуации уважителен в общении с другими.

«30» мая 2026 г.

Отчет проверил:
Руководитель практики от Университета

 (С.П. Луцкан)
(подпись)

Оглавление

Перечень сокращений и обозначений.....	7
Введение.....	8
Раздел 1. Проектная часть	10
1.1. Постановка производственной задачи.....	10
1.2. Роль в проекте и организация работы.....	10
Раздел 2. Практическая часть.....	11
2.1. Аналитический обзор и постановка задачи.....	11
2.2. Проектирование информационной системы.....	12
2.3. Разработка и тестирование.....	14
Раздел 3. Техничко-экономическое обоснование	19
Заключение	21
Список использованных источников	21
Приложение А. Листинги программного кода	23
Приложение Б. Техническое задание.....	27

Перечень сокращений и обозначений

Сокращение	Расшифровка
CV	Computer Vision (Компьютерное зрение)
CRNN	Convolutional Recurrent Neural Network (сверточные рекуррентные нейронные сети)
LSTM	Long Short-Term Memory (долгая краткосрочная память)
CTC Loss	Connectionist Temporal Classification Loss (Коннекционистская временная классификация)
OCR	Optical Character Recognition (Оптическое распознавание символов)
ML	Machine Learning (Машинное обучение)
ОН	Основная надпись
ПО	Программное обеспечение
ИКС	Интерфейс командной строки (CLI)
ОИ	Область интереса (ROI)
ГОСТ	Государственный стандарт
ЕСКД	Единая система конструкторской документации
СПДС	Система проектной документации для строительства
СЭД	Система электронного документооборота

Введение

В эпоху цифровизации инженерной деятельности значительные объемы технической документации, накопленные за десятилетия, переводятся в электронный вид путем сканирования. Однако этот процесс создает лишь графические копии, оставляя нерешенной задачу извлечения и структурирования ценных метаданных, содержащихся в основной надписи (штампе) чертежа. Ручной ввод этих данных – трудоемкий, медленный и подверженный ошибкам процесс. Применение алгоритмов машинного обучения для автоматизации распознавания и извлечения данных из ОН, является актуальной задачей, направленной на повышение эффективности работы с электронными архивами и снижение влияния человеческого фактора.

Объектом исследования являются процессы и алгоритмы обработки изображений для извлечения текстовой информации из графических документов.

Предметом исследования является применение алгоритмов машинного обучения для разработки программного средства, реализующего конвейер для локализации и распознавания основной надписи на отсканированных технических чертежах.

Целью данной технологической практики является разработка программных алгоритмов с применением методов машинного обучения для автоматизации перевода аналоговой документации (чертежей) в цифровой вид. Для достижения поставленной цели были определены следующие задачи:

1. Провести теоретические исследования: изучить проблематику ручной обработки чертежей, проанализировать релевантные государственные стандарты [1, 2, 3, 4] и существующие подходы в области компьютерного зрения и машинного обучения.

2. Применить методы системного анализа: определить границы проекта, выбрать технологический стек и спроектировать архитектуру программного конвейера.

3. Разработать программные модули: реализовать алгоритмы предобработки изображений, локализации ОН и применить готовые ML-модели для извлечения текста.

4. Провести экспериментальные исследования: протестировать работоспособность прототипа на наборе данных, оценить качество его работы.

5. Подготовить техническую документацию, описывающую функциональность, структуру и принципы работы разработанного ПО.

Методы исследования: теоретический анализ научной литературы и стандартов, системный анализ и проектирование ПО, алгоритмизация, программирование с применением ML-библиотек, экспериментальное тестирование, анализ результатов.

Раздел 1. Проектная часть

1.1. Постановка производственной задачи

В ходе прохождения технологической практики в АО «Метрогипротранс» была поставлена задача оптимизации процессов обработки архивной технической документации. В ходе деятельности института возникает необходимость обработки больших массивов отсканированных чертежей. Текущий процесс ввода атрибутивной информации (шифр, номер листа и т.д.) выполняется сотрудниками вручную, что приводит к значительным временным затратам и риску возникновения ошибок из-за человеческого фактора.

Проект направлен на решение данной производственной задачи по автоматизации извлечения атрибутивной информации. Целью проекта является соперации по поиску основной надписи на чертеже и распознаванию ее содержимого, оставляя за сотрудником только функцию финальной проверки (валидации) результатов.

1.2. Роль в проекте и организация работы

Работа выполнялась под руководством начальника отдела и при консультационной поддержке инженеров архивного отдела. В рамках проекта на меня были возложены функции инженера-разработчика, занимающегося исследованием и прототипированием решения. Выполняемые трудовые функции:

- Анализ требований: Взаимодействие с представителями архивного отдела для уточнения требований к извлекаемым данным;
- Исследование технологий: Анализ существующих библиотек и фреймворков в области CV и ML (в частности, OpenCV и EasyOCR);
- Разработка прототипа: Проектирование архитектуры ПО, написание кода, реализующего конвейер обработки;
- Тестирование: Проверка работоспособности прототипа на предоставленных примерах чертежей;
- Документирование: Подготовка технической документации и отчета по практике.

Раздел 2. Практическая часть

2.1. Аналитический обзор и постановка задачи

Проблема автоматизации ввода данных с бумажных носителей является классической для многих отраслей. В инженерной сфере она осложняется строгими требованиями к формату документации, регламентированными стандартами ГОСТ. Решение данной задачи лежит в области компьютерного зрения и применения алгоритмов машинного обучения. Современные подходы к распознаванию текста на изображениях (OCR) основаны на глубоких нейронных сетях, таких как сверточные (CNN) и рекуррентные (RNN) сети. Библиотека EasyOCR, выбранная для данного проекта, использует именно такой подход – архитектура CRNN [5], что позволяет классифицировать ее как инструмент машинного обучения.

Несмотря на высокую точность современных систем распознавания, существуют границы их применимости, которые должны быть четко обозначены в регламентах ПО.

1) Исключение геометрии: Автоматизация касается только метаданных (штампа). Распознавание геометрических примитивов (линий, размеров, арматуры) на чертеже требует совершенно иного уровня сертификации ПО и контроля. Ошибки в распознавании размера на чертеже могут привести к техногенным катастрофам, поэтому данная область остается зоной ответственности инженера-человека.

2) Юридическая значимость: Скан-образ чертежа признается юридически значимым только при наличии электронной подписи или соответствующего акта оцифровки. Автоматически извлеченные метаданные служат лишь для поиска, но не подменяют собой содержание документа.

3) Сложные случаи: Чертежи с наклеенными фрагментами, сильными разрывами бумаги в области штампа или отсутствием стандартизированных граф (в документах начала 1930-х годов) требуют только ручного описания. Система должна уметь вовремя «отказаться» от обработки и передать задачу специалисту.

2.2. Проектирование информационной системы

Архитектура разработанного решения представляет собой последовательный конвейер обработки данных (pipeline), включающий модуль предобработки, модуль локализации и модуль интеллектуального анализа.

1) Коррекция перекоса (Deskew) с помощью преобразования Хафа.

Для выравнивания изображения используется вероятностное преобразование Хафа. Метод преобразует точки на изображении из декартовой системы координат (x, y) в параметрическое пространство (ρ, θ) , где каждая точка соответствует прямой, заданной уравнением в нормальной форме:

$$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta),$$

где ρ – а расстояние от начала координат до прямой, θ – угол перпендикуляра к прямой. Пики в пространстве Хафа (аккумуляторе) соответствуют доминирующим прямым на изображении. В данном проекте анализируются пики, соответствующие горизонтальным линиям, и вычисляется средний угол их наклона, на который затем поворачивается изображение.

2) Фильтрация контуров для локализации ОН.

После бинаризации изображения и поиска всех замкнутых контуров производится их фильтрация по геометрическим критериям:

- Аппроксимация формы: Контур аппроксимируется многоугольником с помощью алгоритма Рамера-Дугласа-Пекера. Для дальнейшего анализа отбираются только контуры, имеющие 4 вершины;

- Фильтр по площади: отсеиваются слишком мелкие и слишком крупные контуры. $S_{min} < S_{contour} < S_{max}$, где S – площадь контура в пикселях;

- Фильтр по соотношению сторон (Aspect Ratio) для оставшихся четырехугольников вычисляется соотношение ширины к высоте $AR = \frac{width}{height}$.

Полученное значение AR сравнивается с эталонными значениями для штампов по ГОСТ (≈ 3.36 для формы 3 и ≈ 12.33 для формы 6) с некоторым допуском ε : $|AR - AR_{gost}| < \varepsilon$.

3) Архитектура нейросети для распознавания текста.

Библиотека EasyOCR использует модель на основе архитектуры CRNN, которая хорошо зарекомендовала себя в задачах распознавания текста на изображениях [6]. Архитектура состоит из трех основных частей:

- Сверточные слои (CNN): выполняют роль извлекателя признаков (feature extractor). На вход подается изображение, и на выходе получается карта признаков;

- Рекуррентные слои (RNN): карта признаков преобразуется в последовательность, которая обрабатывается рекуррентными слоями (обычно LSTM), способными улавливать контекстуальные зависимости в последовательности;

- Слой транскрипции (CTC Loss): это функция потерь, используемая для обучения модели, которая позволяет работать с разметкой "текст на картинке" без необходимости точного позиционирования каждого символа [7]. Логическая последовательность работы алгоритмов представлена на рисунке 1.

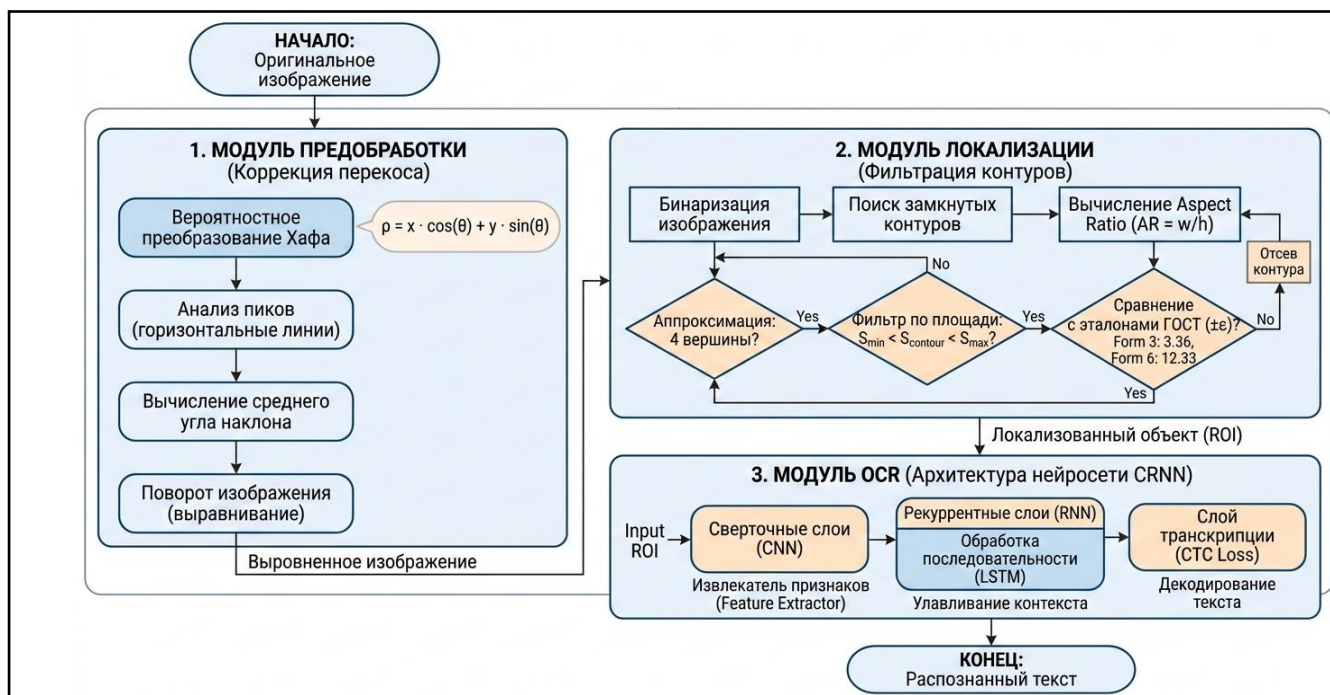


Рисунок 1 – Блок-схема алгоритма локализации области основной надписи.

2.3. Разработка и тестирование

Для реализации прототипа был выбран язык программирования Python и набор библиотек: OpenCV для задач классической обработки изображений и EasyOCR для применения обученных моделей распознавания текста. В ходе разработки особое внимание уделялось модульности кода, чтобы обеспечить возможность дальнейшего расширения и модификации. Для удобства использования был реализован интерфейс командной строки (ИКС).

На рисунке 2 продемонстрирован результат локализации штампа. Зеленым контуром выделена найденная область интереса, соответствующая пропорциям ГОСТ.

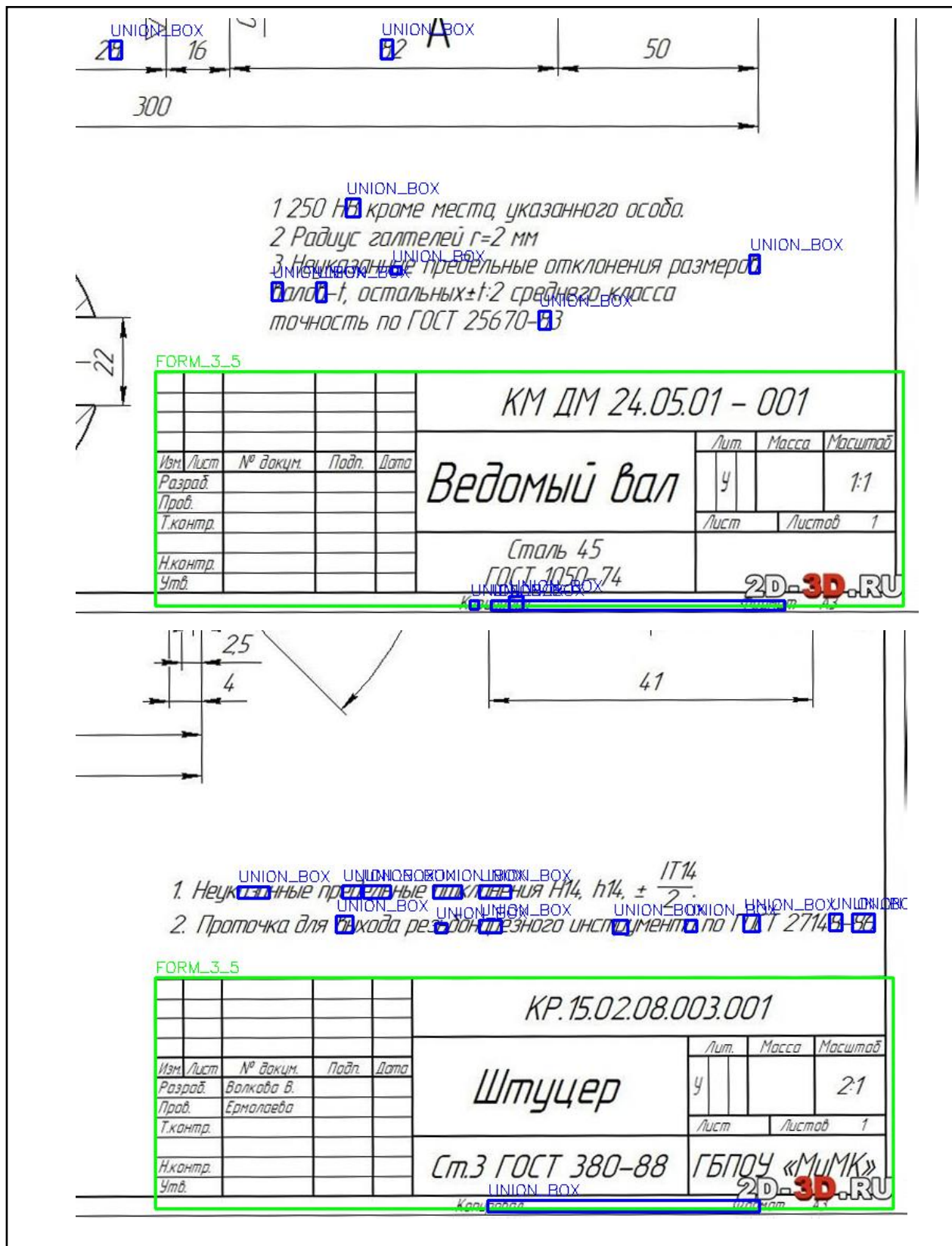


Рисунок 2 – Результат работы алгоритма локализации штампа ROI.

Код проекта был организован в виде пакета `gost_ocr` с четким разделением логики по модулям, что соответствует спроектированной архитектуре:

- cli.py: Точка входа, обработка аргументов командной строки
- config.py: Хранение констант (пути, параметры ГОСТ, пороги)
- preprocessing.py: Модуль предобработки
- localization.py: Модуль локализации штампа
- extraction.py: Модуль извлечения текста.

В области интереса производится поиск контуров. С помощью анализа иерархии и геометрических свойств (соотношение сторон, количество вершин) находится внешний контур штампа. Алгоритм валидирует найденные прямоугольники на соответствие пропорциям стандартных форм ГОСТ.

Внутри найденной рамки штампа используется движок EasyOCR для распознавания всего текста. Основным форматом вывода пакета gost_osr является JSON, который содержит следующую информацию:

- source_image_path: Путь к исходному файлу
- stamp_bounding_box: Координаты найденной рамки штампа
- text_blocks: Массив объектов, где каждый объект представляет распознанный блок текста с его содержимым (text), уверенностью (confidence) и координатами (box)
- full_text: Весь распознанный текст в виде одной строки.

Такая структура позволяет легко интегрировать результаты работы конвейера с другими системами (например, СЭД) или использовать их для дальнейшего анализа. На рисунке 3 показан результат извлечения текста из найденной области интереса (ROI).

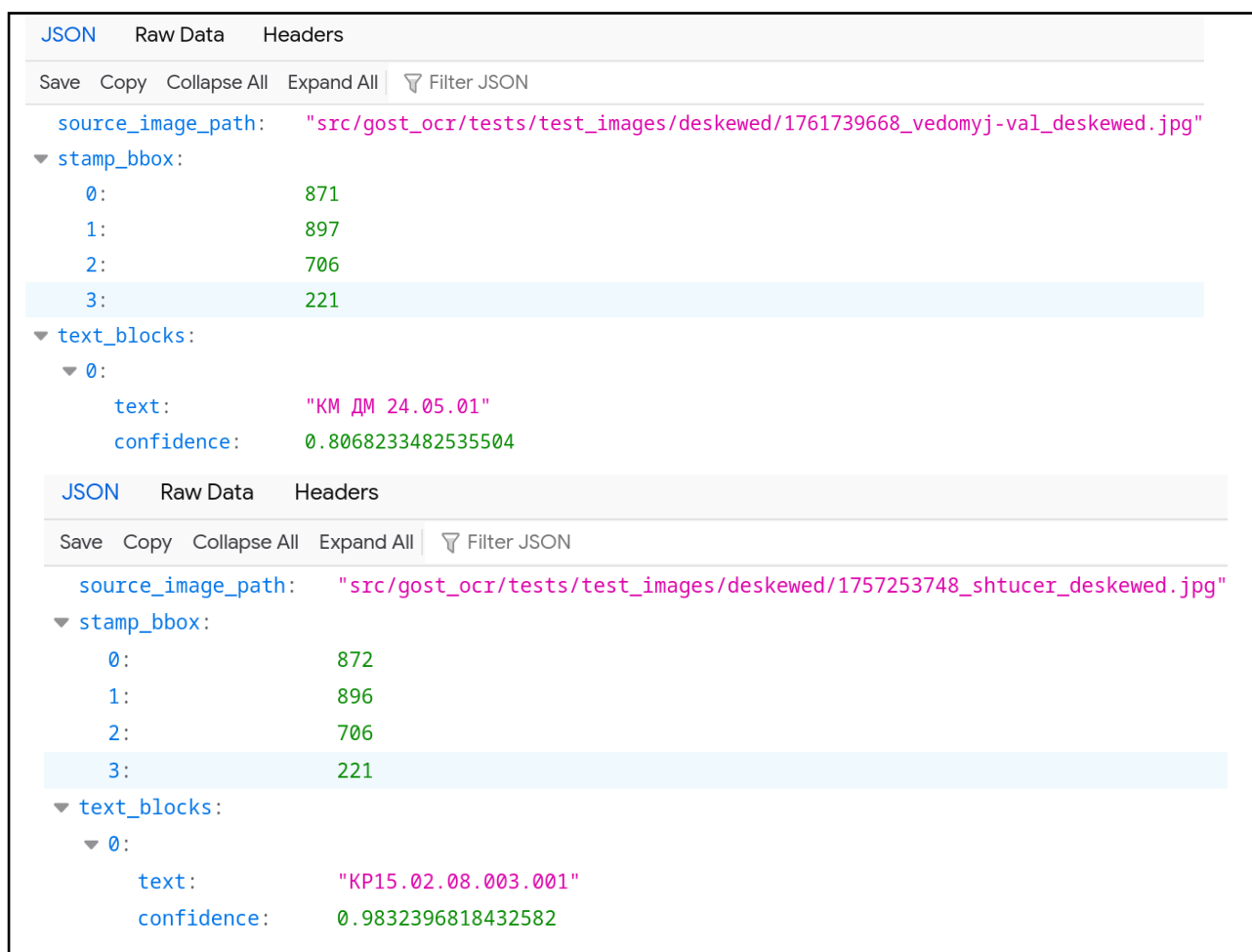


Рисунок 3 – JSON вывод извлеченного текста.

Тестирование проводилось на наборе из 20 изображений-примеров из открытых источников, расположенных в `src/gost_ocr/tests/test_images/`. Эти изображения включают как качественные сканы, так и примеры с поворотом, перекосом и артефактами. Каждый модуль проверялся отдельно. Например, для модуля `localization.py` использовался флаг `--draw-all`, который визуализирует все найденные прямоугольные контуры, позволяя оценить корректность работы алгоритма фильтрации. Использование флага `--debug` позволяет сохранять промежуточные результаты обработки: предобработанное изображение, ROI, изображение с нарисованными контурами в папку `debug/`. Это ключевой инструмент для визуальной проверки и отладки каждого шага конвейера.

Качественная оценка показала, что алгоритм успешно справляется с поиском штампа на документах хорошего и среднего качества. Точность

распознавания текста напрямую зависит от качества исходного скана и возможностей используемой ML-модели. Для проверки корректности работы были применены следующие подходы:

Целью тестирования на данном этапе была не количественная оценка точности (precision/recall), а качественная проверка работоспособности всех компонентов системы и их устойчивости к типичным дефектам сканирования.

Раздел 3. Технико-экономическое обоснование

Целью данного раздела является оценка экономической целесообразности внедрения разработанного программного средства gost-ocr.

1. Расчет трудозатрат на ручную обработку

Исходные данные:

- Среднее время обработки одного чертежа вручную: 5 минут (0.083 ч).
- Средняя месячная зарплата сотрудника архива (условно): 50 000 руб.
- Количество рабочих часов в месяце: 168
- Стоимость одного часа работы сотрудника: $50\,000 / 168 \approx 298$ руб./час.

Стоимость ручной обработки одного листа:

$$C_{manual} = 0.083 \text{ ч} * 298 \text{ руб./ч} \approx 24.73 \text{ руб.}$$

2. Расчет трудозатрат на автоматизированную обработку

Исходные данные:

- Время машинной обработки одного чертежа: ~10 секунд.
- Время на верификацию результата оператором: ~20 секунд.
- Общее время: 30 секунд (0.0083 часа).

Стоимость автоматизированной обработки одного листа:

$$C_{auto} = 0.0083 \text{ ч} * 298 \text{ руб./ч} \approx 2.47 \text{ руб.}$$

3. Оценка экономического эффекта

Экономия на обработке одного листа:

$$E_{sheet} = C_{manual} - C_{auto} = 24.73 - 2.47 = 22.26 \text{ руб.}$$

Предположим, в месяц обрабатывается архив из 2000 листов.

- Затраты при ручной обработке: $2000 * 24.73 = 49\,460$ руб
- Затраты при автоматизированной обработке: $2000 * 2.47 = 4\,940$ руб

Годовой экономический эффект от внедрения системы (при обработке 24 000 листов в год) составит:

$$E_{year} = (49\,460 - 4\,940) * 12 = 44\,520 * 12 = 534\,240 \text{ руб.}$$

Данный расчет является предварительным, но наглядно демонстрирует высокую экономическую эффективность проекта, которая достигается за счет многократного сокращения трудозатрат на рутинные операции.

Заключение

В ходе технологической практики была решена задача разработки и исследования алгоритмов машинного обучения для перевода аналоговой информации в цифровую на примере распознавания основной надписи технических чертежей.

Основные результаты:

1. Разработан прототип программного обеспечения, реализующий конвейер обработки изображений и применяющий ML-модель для распознавания текста.
2. Продемонстрирована принципиальная возможность и высокая экономическая эффективность автоматизации рутинной производственной задачи.
3. Получены практические навыки применения библиотек компьютерного зрения и машинного обучения для решения прикладных задач.

Разработанный прототип может служить основой для дальнейшей разработки полноценного промышленного решения в АО «Метрогипротранс». Дальнейшие шаги могут включать дообучение OCR-модели на специфических данных предприятия и интеграцию ПО в существующую IT-инфраструктуру. Реализация предложенного подхода обеспечит быстрый и точный доступ к историческим данным, что критически важно для безопасной эксплуатации и развития транспортной системы Москвы.

Список использованных источников

1. ГОСТ 2.104-2006. Единая система конструкторской документации. Основные надписи. Введ. 2006-09-01. – М.: Стандартинформ, 2006. – с. 18.
2. ГОСТ Р 21.101-2020. Система проектной документации для строительства. Основные требования к проектной и рабочей документации. – Введ. 2021-01-01. – М.: Стандартинформ, 2020. – с. 62.
3. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения. – Введ. 1992-01-01. – М.: Издательство стандартов, 1991. – с. 26.
4. ГОСТ 19.401-78. Единая система программной документации. Текст программы. Требования к содержанию и оформлению. – Введ. 1980-01-01. – М.: Издательство стандартов, 1979. – с. 5.
5. Yadav, A. OCR using CRNN: A Deep Learning Approach for Text Recognition / A. Yadav, S. Singh, A. Kotangale // 2023 4th International Conference for Emerging Technology (INCET). – 2023. – с. 1-6.
6. Prakash, P. CRNN model for text detection and classification from natural scenes / P. Prakash, S. Hanumanthaiah, S. Mayigowda // IAES International Journal of Artificial Intelligence. – 2024. – Vol. 13, No. 2. – с. 839-849.
7. A Text Recognition Model Based on Multi-Scale Fusion CRNN / J. Lv, Y. Zhang, Q. Liu [et al.] // Electronics. – 2023. – Vol. 12, No. 4. – с. 948.
8. Bradski, G. The OpenCV Library [Электронный ресурс] / G. Bradski // Dr. Dobb's Journal of Software Tools. — 2000. – Режим доступа: <https://opencv.org/> (дата обращения: 25.01.2026).
9. Документация EasyOCR [Электронный ресурс]. – Режим доступа: <https://www.jaided.ai/easyocr/> (дата обращения: 27.01.2026).
10. Документация библиотеки Tysper [Электронный ресурс]. – Режим доступа: <https://typer.tiangolo.com/> (дата обращения: 02.02.2026).

Приложение А. Листинг программного кода.

Модуль предобработки (`src/gost\_ocr/preprocessing.py`)

```
import cv2
import numpy as np

def deskew(image: np.ndarray) -> np.ndarray:
    """Выравнивает изображение, устраняя перекося."""
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    gray = cv2.bitwise_not(gray)
    lines = cv2.HoughLinesP(
        gray, 1, np.pi / 180, 100, minLength=100, maxLineGap=10
    )
    if lines is None:
        return image

    angles = []
    for line in lines:
        x1, y1, x2, y2 = line[0]
        angle = np.rad2deg(np.arctan2(y2 - y1, x2 - x1))
        angles.append(angle)

    # Вычисляем медианный угол и поворачиваем
    median_angle = np.median(angles)
    (h, w) = image.shape[:2]
    center = (w // 2, h // 2)
    rotation_matrix = cv2.getRotationMatrix2D(center, median_angle, 1.0)
    rotated = cv2.warpAffine(
        image,
        rotation_matrix,
        (w, h),
        flags=cv2.INTER_CUBIC,
        borderMode=cv2.BORDER_REPLICATE,
    )
    return rotated
```

Модуль локализации (`src/gost\_ocr/localization.py`)

```
# Содержимое файла src/gost_ocr/localization.py
import cv2
import numpy as np

from gost_ocr.config import GOST_ASPECT_RATIO_TOLERANCE,
GOST_STAMP_ASPECT_RATIOS

def find_stamp_contour(processed_image: np.ndarray) -> np.ndarray | None:
    """Находит контур штампа на предобработанном изображении."""
    contours, hierarchy = cv2.findContours(
        processed_image, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE
    )

    if hierarchy is None:
        return None

    for i, contour in enumerate(contours):
        # Родительский контур без родителя
        if hierarchy[0][i][3] == -1:
            perimeter = cv2.arcLength(contour, True)
            approx = cv2.approxPolyDP(contour, 0.02 * perimeter, True)

            if len(approx) == 4:
                x, y, w, h = cv2.boundingRect(approx)
                if w > 100 and h > 20: # Фильтр по минимальным размерам
                    aspect_ratio = w / float(h)
                    for _, ratio in GOST_STAMP_ASPECT_RATIOS.items():
                        if abs(aspect_ratio - ratio) < GOST_ASPECT_RATIO_TOLERANCE:
                            return approx

    return None
```

Модуль интеллектуального анализа (`src/gost-ocr/ extraction.py`)

```
from __future__ import annotations
import dataclasses
from dataclasses import dataclass, field
from pathlib import Path
import cv2
import easyocr
import numpy as np

from .config import DEBUG_EXTRACTION_DIR
from .localization import LocalizationResult

_ocr_instance = None

def get_ocr_instance() -> easyocr.Reader:
    """Инициализация easyocr.Reader"""
    global _ocr_instance
    if _ocr_instance is None:
        print("Initializing EasyOCR model (ru)...")
        _ocr_instance = easyocr.Reader(["ru"], gpu=False)
        print("EasyOCR model initialized.")
    return _ocr_instance

@dataclass
class TextBlock:
    text: str
    confidence: float
    box: list[list[int]]

@dataclass
class ExtractionResult:
    source_image_path: str
    stamp_bbox: tuple[int, int, int, int]
    text_blocks: list[TextBlock] = field(default_factory=list)
    full_text: str = ""

def extract_text(
```

```

        localization_result: LocalizationResult, debug: bool = False
    ) -> ExtractionResult | None:
        if not localization_result.stamp:
            return None

        stamp_bbox = localization_result.stamp.bbox
        original_image = localization_result.preprocessed.image

        x, y, w, h = stamp_bbox
        x, y, w, h = max(0, x), max(0, y), max(0, w), max(0, h)

        if w == 0 or h == 0:
            return None

        stamp_image = original_image[y : y + h, x : x + w]

        if stamp_image.size == 0:
            return None

        ocr = get_ocr_instance()
        ocr_result = ocr.readtext(stamp_image)

        text_blocks = []
        full_text_lines = []
        if ocr_result:
            for box, text, confidence in ocr_result:
                int_box = [[int(p[0]), int(p[1])] for p in box]
                text_blocks.append(
                    TextBlock(text=text, confidence=float(confidence), box=int_box)
                )
                full_text_lines.append(text)

        return ExtractionResult(
            source_image_path=str(localization_result.preprocessed.original_path),
            stamp_bbox=stamp_bbox,
            text_blocks=text_blocks,
            full_text="\n".join(full_text_lines),
        )

```


Приложение Б. Техническое задание

I. Общие сведения

1. Наименование программы: Программный модуль автоматизированного распознавания основных надписей «GOST-OCR».

2. Назначение: Автоматизация ввода метаданных с отсканированных чертежей в рамках деятельности АО «Метрогипротранс».

3. Основание для разработки: План прохождения технологической практики, производственная необходимость в оптимизации работы архивного отдела.

II. Требования к программе

1. Требования к функциональным характеристикам:

- Программа должна обеспечивать загрузку изображений форматов jpg, png из указанной директории.

- Программа должна выполнять автоматическую предобработку изображений, включая коррекцию перекоса.

- Программа должна выполнять локализацию области основной надписи, соответствующей пропорциям ГОСТ 2.104 и ГОСТ Р 21.101.

- Программа должна выполнять оптическое распознавание текста в выделенной области с помощью интегрированной ML-модели.

- Программа должна сохранять результаты распознавания в структурированном формате JSON.

2. Требования к надежности:

- Программа должна обеспечивать стабильную обработку файлов без аварийного завершения.

- Предусмотреть логирование ошибок в случае невозможности обработки файла.

3. Требования к составу и параметрам технических средств (Hardware):

- Процессор: не ниже Intel Core i3 или аналог.

- Оперативная память (ОЗУ): не менее 4 Гб.

III. Этапы разработки

1. Анализ и проектирование.
2. Разработка модуля предобработки.
3. Разработка модуля локализации.
4. Интеграция OCR-модуля и тестирование.
5. Подготовка отчетной документации.